UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME TÉCNICO – PROYECTO DEL PRIMER PARCIAL

SISTEMA DE GESTIÓN DE RUTAS ÓPTIMAS EN CUSCO: ANÁLISIS COMPARATIVO DE ALGORITMOS GREEDY, DIVIDE Y VENCERÁS, PROGRAMACIÓN DINÁMICA Y BACKTRACKING

Curso:
PROGRAMACIÓN III

Integrantes:
Amilcar Estacio Medrano (200822)
Kevin Heli Choquecahua Gutierrez (245570)
Jeliel Enrique Aramburu (210927)

Docente:
Mg. Karla Catherine Rojas Pedraza

Perú, Octubre de 2026

Lista de Acrónimos

APA	American Psychological Association
B&B	Branch and Bound (Ramificación y Poda)
CVRP	Capacitated Vehicle Routing Problem (Problema de Enrutamiento de Vehículos con Capacidad)
DFS	Depth-First Search (Búsqueda en Profundidad)
DyV	Divide y Vencerás (Divide and Conquer)
EPIIS	Escuela Profesional de Ingeniería Informática y de Sistemas
GPS	Global Positioning System (Sistema de Posicionamiento Global)
GUI	Graphical User Interface (Interfaz Gráfica de Usuario)
JSON	JavaScript Object Notation (Notación de Objetos de JavaScript)
OSM	OpenStreetMap
PD	Programación Dinámica (Dynamic Programming)
TSP	Travelling Salesperson Problem (Problema del Viajante de Comercio)
UNSAAC	Universidad Nacional de San Antonio Abad del Cusco
VRP	Vehicle Routing Problem (Problema de Enrutamiento de Vehículos)
WGS84	World Geodetic System 1984 (Sistema Geodésico Mundial 1984)

Agradecimientos

Agradecemos a la Universidad Nacional de San Antonio Abad del Cusco (UNSAAC) y a la Escuela Profesional de Ingeniería Informática y de Sistemas por brindarnos la formación académica y científica necesaria para el desarrollo de este proyecto.

Expresamos nuestra sincera gratitud a la docente del curso de Programación III, Mg. Karla Catherine Rojas Pedraza, por sus valiosas orientaciones pedagógicas, exigencia metodológica y retroalimentación constante durante el semestre académico. Asimismo, extendemos nuestro reconocimiento a nuestros compañeros de equipo por el compromiso y esfuerzo compartido a lo largo de cada fase de este trabajo.

Índice general

Lista de Acrónimos **II**

Agradecimientos **III**

Índice de Tablas **VII**

Índice de Figuras **VIII**

Resumen **IX**

Abstract **X**

1. Introducción **1**

2. Planteamiento del Problema **3**

2.1. Descripción de la Situación Problemática 3

2.2. Problema General 4

2.3. Problemas Específicos 4

2.4. Justificación 4

3. Objetivos de la Investigación **6**

3.1. Objetivo General 6

3.2. Objetivos Específicos 6

4. Marco Teórico **7**

4.1. Antecedentes 7

4.2. Bases Teóricas 8

4.2.1. Modelo de la red vial 8
4.2.2. Estructuras, ordenación y búsqueda 9
4.2.3. Paradigma Greedy 9
4.2.4. Paradigma Divide y Vencerás 10
4.2.5. Programación Dinámica 11
4.2.6. Backtracking 11
4.2.7. Notación Big-O 12

5. Hipótesis de Investigación **13**
5.1. Hipótesis General 13
5.2. Hipótesis Específicas 13
5.3. Verificación de Hipótesis 14

6. Metodología de la Investigación **15**
6.1. Tipo de Investigación 15
6.2. Proceso Metodológico 15
6.2.1. Fase 1: Modelo de datos y red vial 15
6.2.2. Fase 2: Implementación algorítmica 16
6.2.3. Fase 3: Interfaz gráfica 17
6.2.4. Fase 4: Verificación y *benchmarking* 17
6.3. Instrumentos y Entorno 18

7. Resultados y Discusión **19**
7.1. Resultados 19
7.1.1. Análisis de complejidad 19
7.1.2. Casos de prueba y resultados esperados 19
7.1.3. Ordenación 20
7.1.4. Greedy frente a Held-Karp: tiempo 20
7.1.5. Greedy frente a Held-Karp: calidad 21
7.1.6. Backtracking: todas las rutas y efecto de la poda 21
7.1.7. Mochila 0/1: Programación Dinámica frente a Greedy 23

7.1.8. Divide y Vencerás 24
7.1.9. Verificación de correctitud y estabilidad 24
7.2. Discusión: ¿cuándo usar cada enfoque? 25

8. Conclusiones y Recomendaciones **27**
8.1. Conclusiones 27
8.2. Recomendaciones 28

Referencias **29**

Apéndice y Anexos **30**

Apéndice y Anexos **38**

Índice de Tablas

7.1. Complejidad Big-O de los algoritmos implementados. 19
7.2. Casos de prueba con resultado esperado y obtenido. 20
7.3. Tiempo de ejecución (ms): Greedy frente a Held-Karp. 20
7.4. Distancia total del recorrido (m) y exceso del Greedy sobre el óptimo. . . 21
7.5. Backtracking sin poda (exhaustivo) y con poda en cuatro pares origen–destino. 22
7.6. Valor cargado (S/.) con capacidad de 25 kg. 23
7.7. Divide y Vencerás: una zona por repartidor (30 pedidos). 24

8.1. Lugares de interés del grafo vial de Cusco. 31
8.2. Divide y Vencerás ($K = 5$) frente a Greedy global con un repartidor. . . . 38

Índice de Figuras

7.1. Tiempo de ejecución Greedy vs Held-Karp 21
7.2. Calidad de ruta: Greedy vs Held-Karp 22
7.3. Backtracking con y sin poda 23
7.4. Mochila y Divide y Vencerás 25

8.1. Pestaña Red Vial 33
8.2. Pestaña Pedidos 34
8.3. Pestaña Greedy 35
8.4. Pestaña Divide y Vencerás 35
8.5. Pestaña Programación Dinámica 36
8.6. Pestaña Backtracking 36
8.7. Pestaña Programación Dinámica 37
8.8. Pestaña Comparación 37

Resumen

El presente trabajo desarrolla un sistema de gestión de rutas óptimas para el reparto de pedidos en la ciudad del Cusco, implementado como aplicación de escritorio en Python con interfaz gráfica Tkinter. La red vial se modela como un grafo dirigido de 20 lugares de interés y 64 conexiones cuyos pesos son longitudes reales de calles de OpenStreetMap; las distancias mínimas entre lugares se precalculan con el algoritmo de Dijkstra implementado a mano. Sobre ese modelo se implementaron, sin librerías de optimización externas, los cuatro paradigmas exigidos: (1) *Greedy*, con la heurística del vecino más cercano y la asignación de pedidos de emergencia al repartidor más próximo; (2) *Divide y Vencerás*, que parte la ciudad por la mediana alternando latitud y longitud, tanto en zonas de K pedidos como en una zona por repartidor; (3) *Programación Dinámica*, con la Mochila 0/1 para cargar el vehículo y Held-Karp para el recorrido exacto; y (4) *Backtracking*, que enumera todos los caminos simples entre dos lugares evitando calles cerradas, con poda opcional por cota. Las mediciones confirmaron el análisis Big-O: Held-Karp tardó 22.2 ms con $n = 12$ puntos frente a 0.017 ms del Greedy, mientras que la ruta voraz llegó a ser 24.5 % más larga que la óptima. En el caso del enunciado, de San Blas a Wanchaq, el Backtracking encontró las 27 rutas simples y la más corta (1,778.6 m); la poda conservó esa misma ruta visitando 20 estados en lugar de 146. Dividir los 30 pedidos en tres zonas, una por repartidor, redujo la distancia total de 43.4 km (un solo repartidor voraz) a 30.2 km. El sistema supera 39 pruebas unitarias y una prueba automática de la interfaz que ejecuta todas sus funciones.

Palabras clave: rutas óptimas, algoritmos greedy, divide y vencerás, programación dinámica, backtracking, Held-Karp, mochila 0/1, Dijkstra, Cusco

Abstract

This work presents an optimal route management system for parcel delivery in the city of Cusco, implemented as a Python desktop application with a Tkinter graphical interface. The road network is modelled as a directed graph of 20 points of interest and 64 connections weighted by real street lengths from OpenStreetMap; shortest distances between places are precomputed with a hand-written Dijkstra algorithm. On top of this model, the four required paradigms were implemented without external optimization libraries: (1) *Greedy*, with the nearest-neighbour heuristic and the assignment of urgent orders to the closest courier; (2) *Divide and Conquer*, which splits the city at the median alternating latitude and longitude, both into zones of K orders and into one zone per courier; (3) *Dynamic Programming*, with the 0/1 Knapsack to load the vehicle and Held-Karp for the exact tour; and (4) *Backtracking*, which enumerates every simple path between two places while avoiding closed streets, with optional bound-based pruning. Measurements confirmed the Big-O analysis: Held-Karp took 22.2 ms for $n = 12$ points against 0.017 ms for Greedy, while the greedy tour was up to 24.5 % longer than the optimum. In the case stated by the assignment, from San Blas to Wanchaq, Backtracking found all 27 simple paths and the shortest one (1,778.6 m); pruning kept that same path while visiting 20 states instead of 146. Splitting the 30 orders into three zones, one per courier, reduced the total distance from 43.4 km (a single greedy courier) to 30.2 km. The system passes 39 unit tests and an automated interface test that exercises every feature.

Keywords: optimal routes, greedy algorithms, divide and conquer, dynamic programming, backtracking, Held-Karp, 0/1 knapsack, Dijkstra, Cusco

1. Introducción

El problema de la distribución eficiente de bienes en entornos urbanos es uno de los desafíos logísticos más relevantes tanto en la industria como en el campo del diseño algorítmico. En ciudades como el Cusco, cuya red vial histórica presenta calles angostas, sentidos únicos, zonas peatonales y calles frecuentemente bloqueadas por actividades culturales o de mantenimiento, la planificación manual de rutas de reparto resulta especialmente compleja e ineficiente.

El presente proyecto, desarrollado en el marco del curso de Programación III de la Escuela Profesional de Ingeniería Informática y de Sistemas (EPIIS) de la Universidad Nacional de San Antonio Abad del Cusco (UNSAAC), aborda esta problemática mediante la construcción de un sistema de gestión de rutas óptimas implementado como aplicación de escritorio en Python. El sistema integra una interfaz gráfica de usuario (GUI) basada en Tkinter que permite visualizar la red vial de Cusco —20 lugares de interés unidos por calles reales de OpenStreetMap—, cerrar calles, gestionar pedidos de reparto y ejecutar cuatro paradigmas algorítmicos fundamentales del diseño de algoritmos: *Greedy*, *Divide y Vencerás*, *Programación Dinámica* y *Backtracking*.

El trabajo no se limita a la implementación funcional de los algoritmos, sino que incluye un análisis riguroso de su complejidad computacional temporal y espacial en notación Big-O, así como la ejecución de *benchmarks* empíricos que permiten comparar el rendimiento real de cada paradigma ante diferentes volúmenes de datos. De este modo, el sistema sirve simultáneamente como herramienta práctica de apoyo a la logística urbana y como plataforma de demostración pedagógica de los paradigmas algorítmicos estudiados durante el semestre.

El documento se organiza de la siguiente manera: el Capítulo 2 describe el plantea-

miento del problema y su justificación; el Capítulo 3 establece los objetivos del proyecto; el Capítulo 4 desarrolla el marco teórico de los algoritmos implementados; el Capítulo 5 formula la hipótesis de trabajo; el Capítulo 6 describe la metodología de desarrollo; el Capítulo 7 presenta los resultados obtenidos y su discusión comparativa; y el Capítulo 8 expone las conclusiones y recomendaciones finales.

2. Planteamiento del Problema

2.1 Descripción de la Situación Problemática

La ciudad del Cusco posee una red vial de carácter histórico que impone restricciones singulares a la movilidad urbana. Calles con ancho insuficiente para vehículos de carga, zonas de tráfico restringido en el Centro Histórico, sentidos únicos de circulación, y cierres temporales frecuentes por eventos culturales o trabajos de mantenimiento, hacen que la planificación manual de rutas de distribución sea altamente ineficiente. A esto se suma la creciente demanda de servicios de reparto a domicilio, que ha crecido de manera sostenida en los últimos años tanto en el ámbito nacional como en la región del Cusco.

Las empresas y emprendimientos locales que realizan reparto de pedidos —restaurantes, farmacias, tiendas de abarrotes, comercio electrónico local— carecen, en su mayoría, de herramientas computacionales que les permitan determinar de forma sistemática qué pedidos priorizar cuando la capacidad del vehículo es limitada, y en qué orden visitarlos para minimizar el tiempo y la distancia recorrida. En la práctica, esta decisión se delega al criterio empírico del repartidor, lo que conduce a rutas subóptimas, mayor consumo de combustible, entregas tardías y reducción de la capacidad de atención diaria.

Desde el punto de vista algorítmico, el problema de la selección óptima de pedidos bajo restricción de capacidad corresponde al problema de la Mochila 0/1 (*Knapsack 0/1*), mientras que la determinación del orden de visita óptimo es una instancia del Problema del Viajante de Comercio (*Travelling Salesperson Problem*, TSP). Ambos pertenecen a la clase de problemas NP-difíciles para los cuales la Programación Dinámica ofrece soluciones exactas de complejidad exponencial, y las heurísticas *Greedy* constituyen alternativas prácticas de complejidad polinomial. La incorporación de restricciones de calles bloqueadas introduce la necesidad de técnicas de búsqueda exhaustiva como el *Backtracking*,

mientras que el reparto de la ciudad entre varios repartidores se aborda con el paradigma *Divide y Vencerás*, que divide el conjunto de pedidos en zonas que cada repartidor resuelve por separado.

Este proyecto responde a la necesidad académica de contrastar empíricamente estos cuatro paradigmas sobre un caso de uso concreto y relevante localmente, analizando sus diferencias de rendimiento y calidad de solución.

2.2 Problema General

¿En qué medida el diseño e implementación de un sistema computacional que aplica los paradigmas algorítmicos Greedy, Divide y Vencerás, Programación Dinámica y Backtracking permite gestionar eficientemente las rutas de reparto en la ciudad del Cusco, optimizando la distancia total recorrida y el cumplimiento de restricciones operativas?

2.3 Problemas Específicos

El estudio toma en cuenta los siguientes problemas específicos:

- ¿Cómo afecta la elección del paradigma algorítmico (Greedy vs. Programación Dinámica) a la calidad de la selección de pedidos bajo restricción de capacidad, medida como distancia total de la ruta generada?
- ¿Cuál es la complejidad temporal y espacial real, medida empíricamente, de cada uno de los cuatro algoritmos implementados en función del número de pedidos activos?
- ¿En qué condiciones del grafo vial (número de calles cerradas, distancia entre origen y destino) el algoritmo de Backtracking logra encontrar rutas alternativas viables en tiempo aceptable para un usuario interactivo?

2.4 Justificación

La justificación de este trabajo es de naturaleza académica y práctica. En el plano académico, el proyecto constituye una demostración integral de cuatro de los paradigmas

algorítmicos más importantes del diseño de algoritmos, aplicados sobre un problema de optimización combinatoria real. La implementación desde cero de cada algoritmo —sin recurrir a librerías de optimización externas— permite a los integrantes del equipo comprender en profundidad la estructura y el comportamiento computacional de cada paradigma, lo que resulta esencial para la formación en Ingeniería Informática y de Sistemas.

En el plano práctico, el sistema desarrollado demuestra que es posible construir, con recursos computacionales modestos y en Python, una herramienta de apoyo a la decisión logística para pequeñas y medianas empresas de reparto en Cusco. El grafo de trabajo tiene 20 lugares de interés, pero sus conexiones no son líneas rectas: cada una corresponde a un recorrido por calles reales de OpenStreetMap descargado con perfil vehicular (`drive`), de modo que las distancias reflejan sentidos de circulación y desvíos reales de la ciudad.

La comparación cuantitativa entre algoritmos, respaldada por *benchmarks* empíricos y análisis Big-O, añade valor pedagógico al permitir visualizar de forma directa la diferencia entre complejidad polinomial y exponencial en un contexto aplicado.

3. Objetivos de la Investigación

3.1 Objetivo General

Diseñar e implementar un sistema de escritorio en Python que gestione pedidos de reparto y genere rutas óptimas sobre la red vial de la ciudad del Cusco (modelada con datos de OpenStreetMap), integrando y comparando cuatro paradigmas algorítmicos —Greedy, Divide y Vencerás, Programación Dinámica y Backtracking— mediante análisis de complejidad Big-O y benchmarks empíricos, con el fin de determinar el paradigma más adecuado según el tamaño del problema y las restricciones operativas.

3.2 Objetivos Específicos

- Implementar desde cero: ordenación (Merge Sort, Quick Sort) y búsqueda (tabla hash, búsqueda binaria, índice por sector) de pedidos; Greedy (vecino más cercano y asignación de emergencia); Divide y Vencerás (partición espacial por mediana en zonas de K pedidos y en una zona por repartidor); Programación Dinámica (Mochila 0/1 y Held-Karp); y Backtracking (todas las rutas simples evitando calles cerradas, con poda opcional), verificando su correctitud con pruebas unitarias en `pytest`.
- Derivar la complejidad temporal y espacial Big-O de cada algoritmo y contrastarla con tiempos y resultados medidos en *benchmarks* reproducibles.
- Desarrollar una interfaz gráfica con Tkinter que no se bloquee durante los cálculos y permita probar visualmente cada funcionalidad: mapa de la red, cierre de calles, gestión de pedidos, ejecución de cada algoritmo y comparación de resultados.

4. Marco Teórico

En este capítulo se presentan los fundamentos teóricos que sustentan el diseño e implementación del sistema: el modelo de la red vial, los paradigmas algorítmicos aplicados, el análisis de complejidad de cada algoritmo implementado y la notación empleada.

4.1 Antecedentes

Dantzig and Ramser (1959) formularon el Problema de Enrutamiento de Vehículos (VRP) como generalización del TSP con múltiples vehículos y capacidad limitada, estableciendo las bases formales del problema que este proyecto aborda en escala reducida. La planificación óptima de rutas es NP-difícil en el caso general, lo que motiva el uso combinado de heurísticas y algoritmos exactos de costo exponencial acotado.

Dijkstra (1959) propuso el algoritmo de caminos mínimos desde un origen en grafos con pesos no negativos, que en este proyecto se usa para calcular la distancia por calles entre cada par de lugares de interés.

Held and Karp (1962) propusieron el algoritmo de Programación Dinámica para el TSP que lleva su nombre, reduciendo la complejidad de la fuerza bruta $O(n!)$ a $O(n^2 2^n)$ mediante memoización sobre subconjuntos de nodos visitados.

Rosenkrantz et al. (1977) analizaron heurísticas constructivas para el TSP, entre ellas la del vecino más cercano, base del enfoque Greedy implementado, y demostraron que, en instancias métricas, puede alejarse del óptimo en un factor que crece con $\log n$; en una red dirigida como la de este proyecto ni siquiera esa cota está garantizada.

Cormen et al. (2022) proporcionan el tratamiento formal de los paradigmas implementados y del Teorema Maestro empleado en el análisis de recurrencias. Toth and Vigo (2014) contextualizan la selección y secuenciación de pedidos con capacidad (CVRP)

dentro de la logística moderna.

4.2 Bases Teóricas

4.2.1. Modelo de la red vial

La ciudad se representa con un grafo dirigido y ponderado $G = (V, E)$. Cada vértice $v \in V$ es uno de los 20 lugares de interés (Plaza de Armas, San Blas, Wanchaq, etc.), con latitud, longitud y sector. Cada arco $(u, v) \in E$ es un recorrido por calles reales entre dos lugares, obtenido de OpenStreetMap con perfil vehicular: su peso $w(u, v)$ es la suma de las longitudes de los tramos de calle, en metros, y conserva la polilínea para dibujarlo. Como existen calles de un solo sentido, $w(u, v)$ puede ser distinto de $w(v, u)$, o uno de los dos arcos puede no existir; la red usada tiene $|E| = 64$ arcos dirigidos: 20 pares en ambos sentidos y 24 arcos de un solo sentido.

El grafo se almacena como diccionario de adyacencia `adyacencia[u][v]` $\rightarrow$ arista, que ocupa $O(|V| + |E|)$, recorre los vecinos de un nodo en $O(\deg(u))$ y consulta una arista concreta en $O(1)$ promedio. Se eligió frente a la matriz de adyacencia porque la red urbana es dispersa (en promedio 3.2 arcos de salida por lugar), de modo que la matriz desperdiciaría $O(|V|^2)$ espacio y obligaría a recorrer filas completas para obtener vecinos. Cada arista lleva además un indicador `activa`: cerrar una calle lo pone en falso en ambos sentidos sin modificar la estructura del grafo.

Distancias mínimas (Dijkstra). Los algoritmos de ruta necesitan la distancia por calles entre lugares que no son vecinos directos. Se ejecuta Dijkstra con cola de prioridad (`heapq`) desde cada uno de los $|V|$ lugares, ignorando las calles cerradas:

$$T_{\text{matriz}} = |V| \cdot O\big((|V| + |E|) \log |V|\big), \qquad S_{\text{matriz}} = O(|V|^2). \tag{4.1}$$

El resultado se guarda en caché y se invalida automáticamente cada vez que se cierra o reabre una calle. Con la matriz disponible, la distancia entre dos lugares se consulta en $O(1)$, y por eso los algoritmos siguientes suponen $d(u, v)$ de costo constante.

La distancia en línea recta (fórmula de Haversine) se usa solo para asignar un pedido nuevo, ingresado por coordenadas, al lugar de interés más cercano, y para mostrar

en la interfaz el factor de desvío entre la línea recta y el recorrido por calles.

4.2.2. Estructuras, ordenación y búsqueda

- **Merge Sort**: divide la lista en dos mitades, las ordena recursivamente y las mezcla. $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ en todos los casos; espacio $O(n)$; es estable.

- **Quick Sort** (partición de Lomuto, pivote por mediana de tres): $\Theta(n \log n)$ en promedio y $O(n^2)$ en el peor caso; espacio $O(\log n)$ promedio por la pila de recursión.

- **Búsqueda por ID con tabla hash** (`dict`): construcción $O(n)$, consulta $O(1)$ promedio.

- **Búsqueda binaria** sobre la lista ordenada por ID: $O(\log n)$ comparaciones, espacio $O(1)$.

- **Índice invertido por sector**: `sector` $\rightarrow$ lista de pedidos; construcción $O(n)$ y consulta $O(1)$ por sector.

4.2.3. Paradigma Greedy

Un algoritmo *Greedy* construye la solución paso a paso tomando en cada etapa la mejor decisión local, sin reconsiderar decisiones anteriores (Cormen et al., 2022). Se aplica en tres puntos:

Vecino más cercano. Desde la posición actual, el repartidor visita el pedido pendiente con menor $d(\text{actual}, p)$. En la iteración i se comparan $n - i$ candidatos:

$$T(n) = \sum_{i=0}^{n-1} (n - i) = \frac{n(n+1)}{2} = O(n^2), \qquad S(n) = O(n). \tag{4.2}$$

Pedido de emergencia. Se asigna el pedido urgente al repartidor con capacidad suficiente cuya distancia por calles al destino sea mínima: una comparación por repartidor, $O(R)$ tiempo y $O(1)$ espacio.

Mochila voraz (solo como referencia de comparación). Ordena los pedidos por valor/kg con Merge Sort y los toma mientras quepan: $O(n \log n)$. No garantiza el óptimo

de la mochila 0/1: con pesos $(1,2,3)$, valores $(15,10,30)$ y $W=4$ toma los dos primeros (valor 25), cuando el óptimo es el primero y el tercero (valor 45).

4.2.4. Paradigma Divide y Vencerás

Divide y Vencerás divide el problema en subproblemas del mismo tipo, los resuelve recursivamente y combina sus soluciones (Cormen et al., 2022). Se usa para segmentar la ciudad.

Partición en zonas de a lo sumo K pedidos. Si hay más de K pedidos, se ordenan con Quick Sort por latitud (niveles pares) o longitud (niveles impares), se cortan por la mediana y se procesa cada mitad. Como el trabajo no recursivo es ordenar,

$$T(n) = 2T(n/2) + \Theta(n \log n). \tag{4.3}$$

Con $a = b = 2$, $n^{\log_b a} = n$ y $f(n) = \Theta(n \log n) = \Theta(n^{\log_b a} \log^1 n)$; por la forma extendida del caso 2 del Teorema Maestro, $T(n) = \Theta(n \log^2 n)$. (Si la mediana se obtuviera por selección lineal en lugar de ordenar, $f(n) = \Theta(n)$ y el resultado sería $\Theta(n \log n)$.) Después, un repartidor entra a la zona cuyo pedido más cercano esté a menor distancia y la recorre con vecino más cercano. Con $M = n/K$ zonas, esa elección cuesta $\sum_i (M-i)K = O(M^2K) = O(n^2/K)$ y las rutas locales $M \cdot O(K^2) = O(nK)$, por lo que el total es $O(n^2/K)$ en tiempo y $O(n)$ en espacio.

Una zona por repartidor (caso de uso del enunciado). Con R repartidores, la ciudad se divide en exactamente R zonas: si $r = 1$ la zona es todo el conjunto; si no, se ordena por el eje del nivel, se corta en la proporción $\lfloor r/2 \rfloor : \lceil r/2 \rceil$ y se resuelve cada parte con ese número de zonas. El árbol tiene profundidad $\lceil \log_2 R \rceil$ y en cada nivel se ordenan n elementos:

$$T(n,R) = \Theta(n \log n \cdot \log R). \tag{4.4}$$

Cada zona se asigna al repartidor más cercano mediante un emparejamiento voraz sobre los R^2 pares ordenados por distancia, $O(R^2 \log R + Rn)$, y cada repartidor calcula la ruta *óptima* de su zona con Held-Karp cuando tiene a lo sumo 10 pedidos.

4.2.5. Programación Dinámica

La Programación Dinámica resuelve un problema a partir de subproblemas superpuestos cuyas soluciones se almacenan para no recalcularlas, apoyándose en el principio de optimalidad (Bellman, 1957).

Mochila 0/1. Dados N pedidos con peso w_i y valor v_i, y una capacidad W, se maximiza $\sum v_i$ sujeto a $\sum w_i \le W$. La tabla $dp[i][c]$ guarda el mejor valor usando los primeros i pedidos con capacidad c:

$$dp[i][c] = \begin{cases} dp[i-1][c] & \text{si } w_i > c \\ \max\left(dp[i-1][c],\ v_i + dp[i-1][c-w_i]\right) & \text{si } w_i \le c \end{cases} \tag{4.5}$$

con $dp[0][c] = 0$. La solución se reconstruye recorriendo la tabla hacia atrás: el pedido i se eligió si $dp[i][c] \neq dp[i-1][c]$. Como la tabla se indexa con enteros, los pesos se expresan en unidades de 0.1 kg (escala 10), *redondeando los pesos hacia arriba y la capacidad hacia abajo*, lo que garantiza que la carga elegida nunca supere la capacidad real. La complejidad es $O(N \cdot W)$ en tiempo y espacio, con W la capacidad en décimas de kg: es pseudo-polinomial, porque depende del valor numérico de W; multiplicar la precisión por 10 multiplica las columnas de la tabla y el tiempo por 10.

Held-Karp (TSP exacto). El estado (mask, u) indica el conjunto de puntos ya visitados, codificado en los bits de un entero, y el punto actual u. Con memoización se calcula

$$C(\text{mask}, u) = \min_{v \notin \text{mask}} \left[d(u,v) + C(\text{mask} \cup \{v\}, v) \right], \tag{4.6}$$

con caso base $C(\text{todos}, u) = d(u, 0)$ si se vuelve al inicio o 0 si el recorrido es abierto. Hay $2^n \cdot n$ estados y cada uno evalúa hasta n transiciones: $O(n^2 2^n)$ tiempo y $O(n\,2^n)$ espacio (Held and Karp, 1962).

4.2.6. Backtracking

El Backtracking construye la solución de forma incremental y retrocede al agotar las opciones de una decisión. Para encontrar las rutas de un origen a un destino se usa una búsqueda en profundidad (DFS) sobre caminos simples:

Algorithm 1: Backtracking para todas las rutas simples

```
Data: nodo actual u, camino, visitados, costo acumulado
if u = destino then
    guardar una copia del camino y su costo;
else
    foreach vecino v de u por una calle abierta do
        if v ∉ visitados then
            agregar v a camino y visitados ;                    // avanzar
            BUSCAR(v, ..., costo + w(u, v));
            quitar v de camino y visitados ;                    // retroceder
        end
    end
end
```

Las calles cerradas se excluyen al pedir los vecinos activos, y el conjunto de visitados impide los ciclos. Sin poda, el algoritmo enumera *todas* las rutas simples, como pide el enunciado, y al final se ordenan por distancia para elegir la más corta.

Poda por cota (Branch & Bound). Opcionalmente se abandona una rama en cuanto su costo parcial iguala o supera la mejor ruta completa encontrada. Como los pesos son no negativos, el costo de una rama solo puede crecer al extenderla: una rama podada nunca terminaría en una ruta más corta, por lo que la poda conserva la ruta óptima y solo reduce el trabajo.

Complejidad. En un grafo de grado medio b y caminos de longitud d, el árbol de búsqueda tiene $O(b^d)$ nodos; en el peor caso, un grafo completo, hay $O(V!)$ caminos simples y copiar cada uno cuesta $O(V)$, es decir, $O(V \cdot V!)$. La poda no mejora esta cota asintótica, pero reduce mucho los estados visitados en la práctica. El espacio es $O(V)$ para la pila, el camino y los visitados, más $O(R \cdot V)$ para guardar las R rutas encontradas.

4.2.7. Notación Big-O

$T(n) = O(f(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $T(n) \leq c\, f(n)$ para todo $n \geq n_0$; Θ indica además una cota inferior del mismo orden (Cormen et al., 2022). Las complejidades de este proyecto, de menor a mayor, son: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n \log^2 n)$, $O(n^2)$, $O(N \cdot W)$ (pseudo-polinomial), $O(n^2 2^n)$ y $O(V \cdot V!)$.

5. Hipótesis de Investigación

5.1 Hipótesis General

La implementación de un sistema computacional que integra los paradigmas algorítmicos Greedy, Divide y Vencerás, Programación Dinámica y Backtracking permite gestionar rutas de reparto en la ciudad del Cusco de forma sistemática y eficiente, con la condición de que el número de pedidos activos no supere 12 para que el algoritmo exacto Held-Karp sea ejecutable en tiempo interactivo (menor a 50 ms en hardware de escritorio estándar).

5.2 Hipótesis Específicas

- El algoritmo de Programación Dinámica Held-Karp producirá, en todos los casos evaluados, una distancia de ruta menor o igual a la del Greedy, con un gap de suboptimalidad del Greedy que crecerá con n, superando el 10 % a partir de $n \geq 8$ pedidos activos.

- El tiempo de ejecución medido empíricamente del algoritmo Held-Karp crecerá de forma exponencial con el número de pedidos n, mientras que el tiempo del algoritmo Greedy crecerá de forma polinomial, verificando los órdenes teóricos $O(2^n \cdot n^2)$ y $O(n^2)$, respectivamente.

- El algoritmo de Backtracking encontrará todas las rutas alternativas simples entre el nodo origen y destino en el grafo vial, aplicando poda de ciclos eficientemente, con un tiempo de ejecución menor a 2 ms para los pares de nodos evaluados en el grafo de 20 nodos de Cusco.

5.3 Verificación de Hipótesis

Tras la ejecución de los *benchmarks* sobre la red vial de OpenStreetMap (Capítulo 7), las hipótesis se verificaron de la siguiente manera:

- **Hipótesis general — Confirmada:** Held-Karp resolvió $n = 12$ puntos en 22.2 ms, por debajo del umbral de 50 ms, y la interfaz lo ejecuta en un hilo de fondo, por lo que la ventana no se bloquea aun con $n = 14$.

- **Hipótesis 1 — Confirmada en lo esencial:** Held-Karp produjo en todos los casos una distancia menor o igual a la del Greedy, y el gap superó el 10 % desde $n = 8$ (24.53 %, 21.23 % y 15.84 % para $n = 8$, 10 y 12). Sin embargo, el gap no creció de forma monótona con n: alcanzó su máximo en $n = 8$. El gap depende de la disposición concreta de los puntos y no solo de su cantidad.

- **Hipótesis 2 — Confirmada:** el tiempo de Held-Karp pasó de 0.059 ms ($n = 5$) a 0.929 ms ($n = 8$), 7.689 ms ($n = 10$) y 22.166 ms ($n = 12$), es decir, se multiplicó por 376 al pasar de 5 a 12 puntos, mientras que el Greedy se mantuvo por debajo de 0.05 ms en todos los casos.

- **Hipótesis 3 — Parcialmente confirmada:** en modo exhaustivo (sin poda) el Backtracking enumeró todas las rutas simples en tres de los cuatro pares (1, 27 y 152 rutas) en menos de 1 ms. En el par extremo N05→N19 alcanzó el tope de seguridad de 500 rutas en 4.1 ms, por encima de los 2 ms previstos. Con la poda activada, los cuatro pares terminaron en menos de 1 ms y con la misma ruta óptima que el modo exhaustivo.

Los tiempos se midieron en el entorno descrito en el Capítulo 6. En otro equipo los valores absolutos cambian, pero no el orden de crecimiento ni las conclusiones; pueden regenerarse con `python benchmarks/benchmark.py`.

6. Metodología de la Investigación

En este capítulo se describe el enfoque metodológico y el procedimiento seguido para diseñar, implementar, verificar y evaluar el sistema.

6.1 Tipo de Investigación

El trabajo es una investigación **aplicada y experimental**. Es aplicada porque resuelve un problema concreto —la planificación de rutas de reparto en Cusco— con conocimientos de algoritmia. Es experimental porque se diseñaron *benchmarks* reproducibles para medir el comportamiento de cada algoritmo y contrastarlo con su análisis Big-O.

6.2 Proceso Metodológico

6.2.1. Fase 1: Modelo de datos y red vial

Se definieron las entidades en `core/modelos.py` con `dataclasses`: `Nodo` (id, nombre, latitud, longitud, sector), `Arista` (origen, destino, longitud en metros, geometría, nombres de calles y estado `activa`), `Pedido` (id, cliente, lugar de destino, coordenadas, sector, prioridad 1–3, peso, volumen y valor), `Repartidor` (vehículo, lugar de inicio y capacidades de peso y volumen) y `Ruta`. La clase `Grafo` guarda los nodos en un diccionario `id` $\rightarrow$ nodo y la adyacencia como diccionario de diccionarios.

La red vial se construyó fuera de línea con `tools/construir_red_vial.py`: se descargó la red de calles de Cusco de OpenStreetMap con perfil vehicular (`network_type="drive"`, respetando sentidos de circulación), se ajustó cada uno de los 20 lugares a la intersección más cercana, y cada lugar se conectó con los 3 lugares más cercanos por calles; se añadieron 4 conexiones más para que todo lugar sea alcanzable desde cualquier otro (grafo fuertemente conexo). El peso de cada conexión es la longitud real del camino por calles.

El resultado se guardó en `data/red_vial_cache.json` (descarga del 19 de octubre de 2026; datos © OpenStreetMap contributors, licencia ODbL), de modo que la aplicación no necesita Internet ni librerías de grafos para funcionar. Ocho lugares quedaron a más de 60 m de su coordenada original al ajustarse a la intersección más cercana (el mayor, el acceso a Sacsayhuamán, a 268 m); el Anexo B lista los ajustes.

Los datos de prueba son 30 pedidos (`data/pedidos.json`) distribuidos en los cuatro sectores, con pesos de 1 a 15 kg y prioridades 1 a 3, y 3 repartidores con bicicleta (15 kg), motocicleta (35 kg) y furgoneta (90 kg) que parten de la Plaza de Armas, Av. El Sol y Marcavalle, respectivamente.

6.2.2. Fase 2: Implementación algorítmica

Los algoritmos se implementaron en `core/`, sin interfaz gráfica, para poder probarlos de forma aislada:

- `core/grafo.py`: Dijkstra con `heapq`, matriz de distancias mínimas en caché y reconstrucción de caminos.
- `core/estructuras.py`: Merge Sort, Quick Sort, búsqueda binaria, índice hash por ID e índice invertido por sector.
- `core/greedy.py`: vecino más cercano, asignación de emergencia y mochila voraz (de referencia).
- `core/divide_venceras.py`: partición espacial en zonas de K pedidos y en R zonas, con asignación de una zona por repartidor.
- `core/prog_dinamica.py`: Mochila 0/1 con tabla y reconstrucción, y Held-Karp con máscara de bits.
- `core/backtracking.py`: búsqueda exhaustiva de rutas simples con poda opcional por cota.

No se usaron librerías de optimización ni de grafos (`networkx`, `scipy`) en la aplicación, ni las funciones `sorted()` o `list.sort()` de Python para las ordenaciones evaluadas. Cada

función documenta en su *docstring* la idea del algoritmo y su complejidad temporal y espacial.

6.2.3. Fase 3: Interfaz gráfica

La GUI usa `tkinter/ttk` con un `Notebook` de siete pestañas: Red Vial, Pedidos, Greedy, Divide y Vencerás, Programación Dinámica, Backtracking y Comparación Big-O. El mapa usa `tkintermapview` con tiles de OpenStreetMap cuando hay conexión; si no la hay, la aplicación lo detecta al iniciar y dibuja la red en un `Canvas` vectorial propio, en el que también se pueden cerrar calles con un clic. Para cumplir el criterio de estabilidad:

- Los cálculos potencialmente largos (Held-Karp, Backtracking y el *benchmark* en vivo) se ejecutan en un hilo de fondo que entrega el resultado por una cola; el hilo principal la revisa cada 50 ms, de modo que ningún hilo secundario toca la interfaz (Tkinter no es seguro entre hilos) y la ventana sigue respondiendo.

- Toda excepción dentro de un botón se muestra en una ventana de aviso y la aplicación continúa funcionando.

- Las entradas del usuario se validan (peso y volumen positivos, prioridad 1–3, coordenadas en rango) y los IDs de pedidos nuevos son únicos.

- Las pestañas comparten un único grafo y una única lista de pedidos: al entrar a una pestaña, su mapa se redibuja si se cerró una calle en otra, y sus listas incluyen los pedidos registrados después.

- Los paneles laterales se desplazan verticalmente para pantallas de 1366×768.

6.2.4. Fase 4: Verificación y *benchmarking*

Pruebas unitarias. `tests/` contiene 39 pruebas `pytest` con resultados esperados conocidos (Anexo C), entre ellas: el caso clásico de mochila cuyo óptimo es 45; la comparación de la Mochila por PD contra fuerza bruta sobre los 2^{12} subconjuntos de 12 pedidos reales; Held-Karp sobre matrices asimétricas con óptimo calculado a mano; el Backtrac-

king en un grafo en forma de diamante con y sin calles cerradas; y las 27 rutas de San Blas a Wanchaq.

Prueba de la interfaz. `tools/prueba_gui.py` abre la aplicación y ejecuta automáticamente cada funcionalidad de cada pestaña (ordenar, buscar, registrar un pedido con datos inválidos y válidos, cada algoritmo, cerrar calles y el *benchmark* en vivo), verificando que no ocurran errores ni bloqueos. Las capturas del Anexo D se generaron con esa prueba.

Benchmarks. `benchmarks/benchmark.py` mide, con `time.perf_counter()`: Merge Sort frente a Quick Sort (promedio de 20 repeticiones); Greedy frente a Held-Karp para $n \in \{3,5,8,10,12\}$; Backtracking con y sin poda en cuatro pares origen–destino; Mochila por PD frente a la voraz para N de 5 a 30 pedidos y 25 kg; Divide y Vencerás frente a Greedy con 30 a 480 pedidos sintéticos (semilla fija); y la división en una zona por repartidor con los 30 pedidos reales. Los resultados se guardan en `benchmarks/resultados_benchmark.json`, del que `informe/generar_figuras.py` genera las figuras de este informe.

6.3 Instrumentos y Entorno

- Lenguaje: Python 3.10 o superior (desarrollo y validación en Python 3.13).
- Dependencias de la aplicación: `tkintermapview`, `matplotlib`, `Pillow`; pruebas con `pytest`. Solo para reconstruir la red vial: `osmnx` y `networkx`.
- Control de versiones: Git y GitHub.
- Entorno de las mediciones reportadas: procesador Intel Xeon a 2.10 GHz (2 núcleos), Linux, Python 3.13. Los tiempos absolutos varían según el equipo; el orden de crecimiento y las comparaciones entre algoritmos se mantienen.

7. Resultados y Discusión

7.1 Resultados

7.1.1. Análisis de complejidad

La Tabla 7.1 resume la complejidad derivada en el Capítulo 4 para cada algoritmo implementado. n es el número de pedidos o puntos, V y E el número de lugares y arcos, W la capacidad en décimas de kg, R el número de repartidores, K el tamaño máximo de zona, y b y d el grado medio y la longitud de los caminos explorados.

Tabla 7.1
Complejidad Big-O de los algoritmos implementados.

Algoritmo	Paradigma	Tiempo	Espacio
Merge Sort	Divide y Vencerás	$\Theta(n \log n)$	$O(n)$
Quick Sort	Divide y Vencerás	$O(n \log n)$ prom., $O(n^2)$ peor	$O(\log n)$
Búsqueda binaria	Disminuir y Vencer	$O(\log n)$	$O(1)$
Búsqueda por ID / sector	Tabla hash	$O(1)$ prom.	$O(n)$
Dijkstra desde cada lugar	Greedy	$O(V(V+E)\log V)$	$O(V^2)$
Vecino más cercano	Greedy	$O(n^2)$	$O(n)$
Asignación de emergencia	Greedy	$O(R)$	$O(1)$
Partición en zonas de K	Divide y Vencerás	$\Theta(n \log^2 n)$	$O(n)$
Ruta por zonas de K	DyV + Greedy	$O(n^2/K)$	$O(n)$
Partición en R zonas	Divide y Vencerás	$\Theta(n \log n \log R)$	$O(n)$
Mochila 0/1	Prog. Dinámica	$O(N \cdot W)$	$O(N \cdot W)$
Held-Karp (TSP)	Prog. Dinámica	$O(n^2 2^n)$	$O(n\,2^n)$
Rutas simples	Backtracking	$O(b^d)$; peor $O(V \cdot V!)$	$O(V)$

7.1.2. Casos de prueba y resultados esperados

La Tabla 7.2 muestra los casos de prueba principales, con el resultado esperado calculado de forma independiente (a mano o por fuerza bruta) y el resultado obtenido. Todos forman parte de las 39 pruebas automáticas del Anexo C.

Tabla 7.2
Casos de prueba con resultado esperado y obtenido.

Algoritmo	Entrada	Esperado	Obtenido
Mochila 0/1	pesos (1,2,3), valores (15,10,30), $W = 4$	valor 45 (ítems 1 y 3)	✓
Mochila 0/1	12 pedidos reales, $W = 20$ kg	óptimo de los 2^{12} subconjuntos	✓
Mochila 0/1	15 pedidos reales, $W = 5$ kg	carga ≤ 5 kg (antes: 5.2 kg)	4.2 kg
Mochila voraz	pesos (1,2,3), $W = 4$	valor 25 (subóptimo)	✓
Held-Karp	matriz asimétrica 3×3	0→2→1→0, 20 m	✓
Held-Karp por zona	8 pedidos reales	distancia $\leq$ Greedy	✓
Backtracking	diamante, sin cierres	2 rutas; óptima por N03 (5 m)	✓
Backtracking	diamante, N01–N03 cerrada	1 ruta, por N02	✓
Backtracking	San Blas → Wanchaq	27 rutas; óptima 1,778.62 m	✓
Backtracking con poda	San Blas → Wanchaq	misma óptima, menos estados	20 vs 146
D&C en R zonas	6 pedidos, $R = 1.,4$	R zonas, tamaños ± 1	✓
D&C por repartidor	30 pedidos, 3 repartidores	3 zonas, 1 por repartidor	✓
Greedy emergencia	pedido urgente, 3 repartidores	el más cercano con capacidad	✓
Búsqueda binaria	ID existente / inexistente	índice / -1	✓

7.1.3. Ordenación

Con 10, 20 y 30 pedidos, Merge Sort tardó 0.011, 0.023 y 0.059 ms, y Quick Sort 0.010, 0.022 y 0.032 ms (promedio de 20 repeticiones). Con tamaños tan pequeños ambos son prácticamente instantáneos; Quick Sort resulta algo más rápido por trabajar en el mismo arreglo, mientras que Merge Sort conserva el orden relativo de los empates (es estable), lo que mantiene el orden de llegada entre pedidos de igual prioridad.

7.1.4. Greedy frente a Held-Karp: tiempo

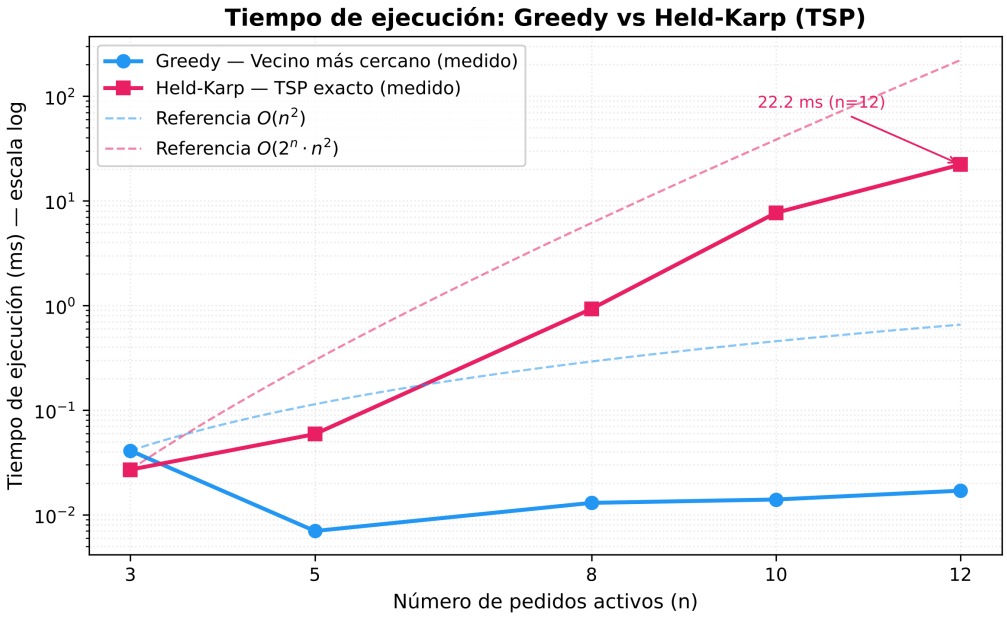
Tabla 7.3
Tiempo de ejecución (ms): Greedy frente a Held-Karp.

n	Greedy (ms)	Held-Karp (ms)	Held-Karp / Greedy
3	0.041	0.027	0,7×
5	0.007	0.059	8,4×
8	0.013	0.929	71,5×
10	0.014	7.689	549×
12	0.017	22.166	1,304×

La Tabla 7.3 y la Figura 7.1 muestran que el tiempo de Held-Karp crece expo-

nencialmente, mientras que el del Greedy apenas varía. Con $n \leq 5$ ambos tiempos son de microsegundos y están dominados por el costo fijo de llamar a una función en Python, por lo que la relación entre ellos no es significativa.

Figura 7.1
Tiempo medido (ms, escala logarítmica) en función de n. Las líneas punteadas son las curvas teóricas $O(n^2)$ y $O(n^2 2^n)$ escaladas al primer punto.



7.1.5. Greedy frente a Held-Karp: calidad

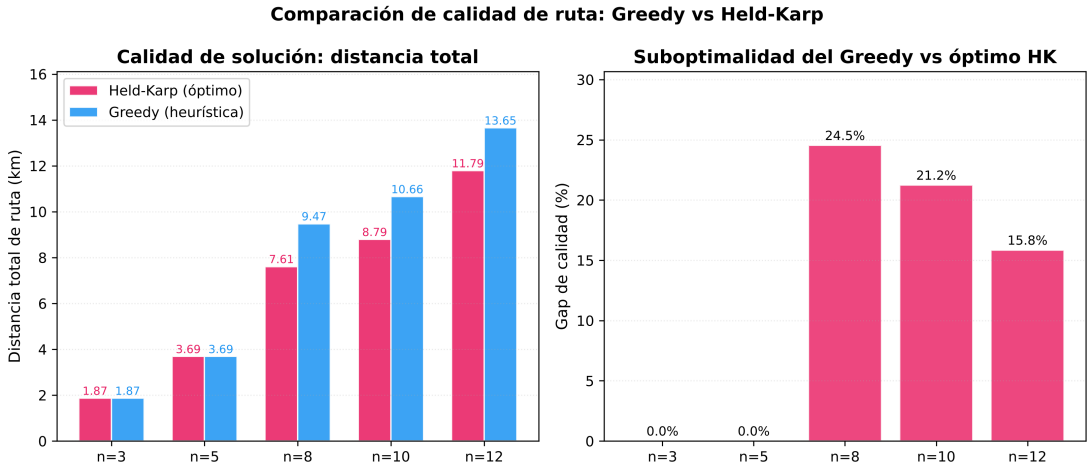
Tabla 7.4
Distancia total del recorrido (m) y exceso del Greedy sobre el óptimo.

n	Held-Karp (m)	Greedy (m)	Exceso (%)
3	1,866.35	1,866.35	0.00
5	3,685.04	3,685.04	0.00
8	7,608.16	9,474.50	24.53
10	8,790.04	10,656.39	21.23
12	11,785.27	13,651.62	15.84

7.1.6. Backtracking: todas las rutas y efecto de la poda

En el caso del enunciado, de San Blas (N02) a Wanchaq (N11) con todas las calles abiertas, la búsqueda exhaustiva encontró **27 rutas simples**, de 1,778.62 m a 35,284.11 m. La más corta es N02→N08→N10→N11 (1,778.62 m); la segunda, N02→N09→N11, mide

Figura 7.2
Izquierda: distancia total del recorrido (km). Derecha: exceso porcentual del Greedy sobre el óptimo.



solo 12.55 m más. Al cerrar por mantenimiento la calle San Blas–Limacpampa (N02–N08), quedan 25 rutas y el algoritmo elige N02→N09→N11 (1,791.17 m), sin pasar nunca por la calle cerrada.

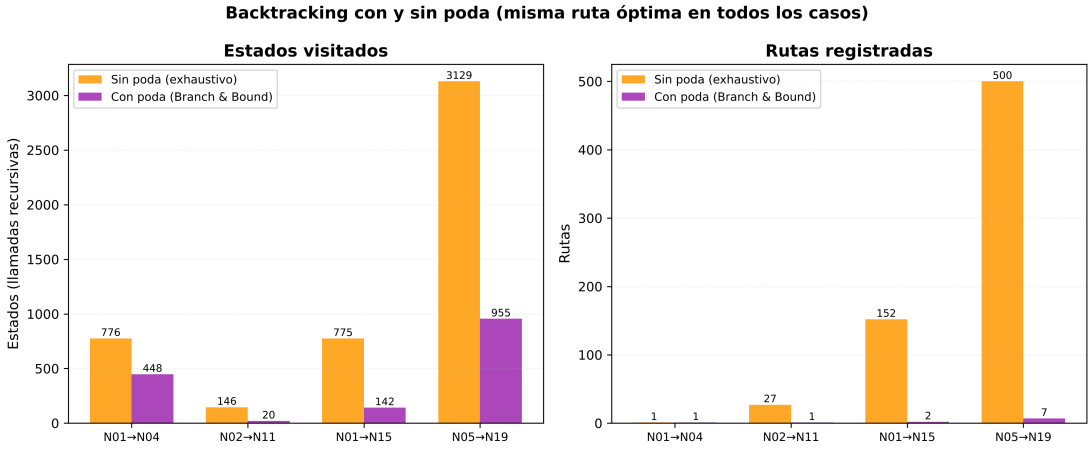
Tabla 7.5
Backtracking sin poda (exhaustivo) y con poda en cuatro pares origen–destino.

	Sin poda			Con poda			
Par	**Rutas**	**Estados**	**ms**	**Rutas**	**Estados**	**ms**	**Óptima (m)**
N01→N04	1	776	0.71	1	448	0.41	790.80
N02→N11	27	146	0.19	1	20	0.02	1,778.62
N01→N15	152	775	0.92	2	142	0.18	4,524.34
N05→N19	500*	3,129	4.09	7	955	0.73	9,034.33

* Alcanzó el tope de seguridad de 500 rutas; con poda se obtuvo la misma óptima.

La Tabla 7.5 y la Figura 7.3 comparan ambos modos. En los cuatro pares la ruta óptima con poda coincide con la del modo exhaustivo, como garantiza el argumento del Capítulo 4, y los estados visitados se reducen entre 42 % y 86 %. Con poda, la columna *Rutas* cuenta solo las rutas que fueron mejorando la mejor distancia conocida: por eso, si se necesita la lista completa de alternativas (por ejemplo, para ofrecer una ruta de reserva), la poda debe desactivarse. El par N01→N04 muestra que el número de estados no depende solo del número de rutas: hay una única ruta simple, pero la red dirigida obliga a explorar 776 estados en callejones sin salida.

Figura 7.3
Estados visitados (izquierda) y rutas registradas (derecha) por el Backtracking sin poda y con poda.



7.1.7. Mochila 0/1: Programación Dinámica frente a Greedy

Con capacidad de 25 kg, la Mochila por PD obtuvo un valor mayor o igual al de la heurística voraz por valor/kg en todos los casos (Tabla 7.6): igual con $N = 5$, 10, 25 y 30, y superior con $N = 15$ (S/. 1,640 frente a S/. 1,625) y $N = 20$ (S/. 1,700 frente a S/. 1,695). La tabla de PD llegó a 7,781 celdas y su cálculo nunca superó 0.7 ms.

Tabla 7.6
Valor cargado (S/.) con capacidad de 25 kg.

N	PD	Voraz	Celdas de la tabla	PD (ms)	Voraz (ms)
5	685	685	1,506	0.131	0.017
10	1,215	1,215	2,761	0.233	0.018
15	1,640	1,625	4,016	0.306	0.026
20	1,700	1,695	5,271	0.427	0.037
25	1,930	1,930	6,526	0.492	0.043
30	2,365	2,365	7,781	0.602	0.058

Durante la revisión se detectó y corrigió un error en la discretización de pesos: la versión inicial redondeaba cada peso al kilogramo más cercano (2.2 kg contaba como 2 kg y 4.5 kg como 4 kg), y con los 15 primeros pedidos llegó a elegir 5.2 kg para una capacidad de 5 kg y 15.2 kg para una de 15 kg. La versión final trabaja con precisión de 0.1 kg, redondea los pesos hacia arriba y la capacidad hacia abajo, y una prueba automática verifica que la carga nunca exceda la capacidad.

7.1.8. Divide y Vencerás

Una zona por repartidor. Con los 30 pedidos y los 3 repartidores, la partición produjo tres zonas de 10 pedidos y cada repartidor calculó con Held-Karp la ruta óptima de su zona (Tabla 7.7). Entre los tres recorren 30.22 km, frente a 43.35 km de un único repartidor que atiende todo con el Greedy; además trabajan en paralelo, por lo que el reparto termina cuando acaba la ruta más larga (12.46 km).

Tabla 7.7
Divide y Vencerás: una zona por repartidor (30 pedidos).

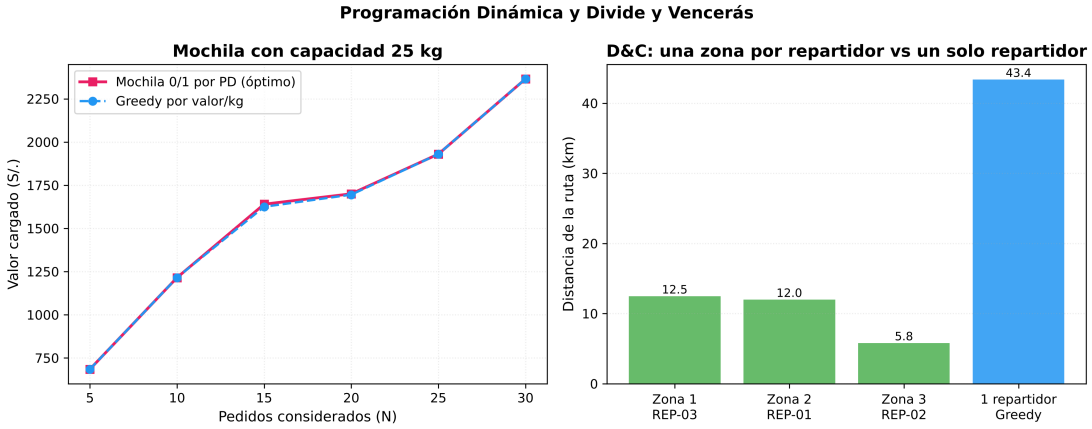
Zona	Repartidor	Pedidos	Ruta óptima (m)
1	REP-03 (furgoneta, Marcavalle)	10	12,457.73
2	REP-01 (bicicleta, Plaza de Armas)	10	11,974.11
3	REP-02 (motocicleta, Av. El Sol)	10	5,788.10
Total del equipo			30,219.94
Un solo repartidor (Greedy, desde la Plaza de Armas)			43,352.76

Un repartidor y zonas de K pedidos. Con pedidos sintéticos (Figura 7.4, derecha, y Anexo E), el tiempo de Divide y Vencerás con $K = 5$ pasó de 0.22 ms ($n = 30$) a 11.6 ms ($n = 480$), similar al del Greedy global. Sin embargo, su ruta fue más larga para $n \geq 60$: como los pedidos se concentran en solo 20 lugares, el corte por coordenadas separa en zonas distintas pedidos de un mismo lugar y obliga a volver a él, mientras que el Greedy los atiende todos de una vez. Para un único repartidor, la segmentación sirve para organizar el trabajo, no para acortar la ruta; su ventaja aparece al repartir zonas entre varios repartidores.

7.1.9. Verificación de correctitud y estabilidad

Las 39 pruebas unitarias de `tests/` se aprobaron sin fallos (Anexo C). La prueba automática de la interfaz ejecutó todas las funcionalidades de las siete pestañas, incluido el *benchmark* en vivo, sin errores ni bloqueos; las capturas del Anexo D proceden de esa ejecución.

Figura 7.4
Izquierda: valor cargado por la Mochila con PD y por la heurística voraz. Derecha: rutas de las tres zonas frente a un único repartidor voraz.



7.2 Discusión: ¿cuándo usar cada enfoque?

Greedy frente a Programación Dinámica en rutas. Held-Karp garantiza el recorrido óptimo, pero por su costo $O(n^2 2^n)$ su tiempo se multiplicó por 376 entre $n = 5$ y $n = 12$, hasta 22 ms; el Greedy respondió siempre en menos de 0.05 ms, pero produjo recorridos hasta 24.5 % más largos. Por tanto, conviene PD cuando el número de paradas es pequeño ($n \leq 12$–14) y el ahorro de distancia importa, por ejemplo en la ruta del día de cada repartidor o dentro de cada zona de Divide y Vencerás. Conviene el Greedy cuando hay muchas paradas, cuando la respuesta debe ser inmediata (pedidos de emergencia) o cuando la ruta se recalcula continuamente.

Greedy frente a Programación Dinámica en la carga. En la mochila la PD resulta casi siempre preferible: su costo $O(N \cdot W)$ es pseudo-polinomial y, con capacidades reales de vehículos, la tabla tiene unos pocos miles de celdas y se calcula en menos de 1 ms. La heurística voraz solo tendría sentido con capacidades enormes o precisión muy fina, donde W haría crecer demasiado la tabla.

Backtracking. Es el único enfoque que enumera *todas* las alternativas y admite cualquier restricción (calles cerradas, sentidos, tramos obligatorios). Su costo es exponencial: en la red de 20 lugares se mantiene por debajo de 5 ms, pero el par N05→N19 ya supera las 500 rutas simples. Conviene cuando el grafo es pequeño o la zona de búsqueda está acotada, cuando se necesitan rutas de reserva, o con la poda activada cuando solo

interesa la ruta óptima bajo restricciones.

Divide y Vencerás. Su aporte es dividir el trabajo: al asignar una zona a cada repartidor, cada subproblema queda lo bastante pequeño para resolverse de forma exacta, y el equipo recorre 30 % menos distancia que un único repartidor. Para un solo vehículo no mejora la calidad de la ruta.

En conjunto, los paradigmas no compiten sino que se complementan. El sistema combina Divide y Vencerás para repartir zonas, PD para cargar cada vehículo y ordenar cada zona, Greedy para las decisiones urgentes y Backtracking cuando las restricciones de la red vial obligan a buscar alternativas.

8. Conclusiones y Recomendaciones

8.1 Conclusiones

- **Sobre la implementación algorítmica:** se implementaron desde cero, sin librerías de optimización ni de grafos, los cuatro paradigmas exigidos y aplicados a los casos de uso del enunciado: Greedy para la ruta voraz y la asignación de emergencias; Divide y Vencerás para segmentar la ciudad en una zona por repartidor; Programación Dinámica para cargar el vehículo (Mochila 0/1) y obtener el recorrido exacto (Held-Karp); y Backtracking para encontrar todas las rutas de San Blas a Wanchaq evitando calles cerradas. Su correctitud está respaldada por 39 pruebas unitarias con resultados esperados calculados de forma independiente.

- **Sobre la complejidad:** las mediciones concuerdan con el análisis Big-O. El tiempo de Held-Karp, $O(n^2 2^n)$, se multiplicó por 376 entre 5 y 12 puntos, mientras que el Greedy, $O(n^2)$, se mantuvo por debajo de 0.05 ms. El análisis también corrigió dos estimaciones iniciales: la partición de Divide y Vencerás cuesta $\Theta(n \log^2 n)$, no $\Theta(n \log n)$, porque en cada nivel se ordena; y su variante con un solo repartidor cuesta $O(n^2/K)$ por la elección voraz entre zonas.

- **Sobre la calidad de las soluciones:** el Greedy produjo recorridos hasta 24.5 % más largos que el óptimo, y la mochila voraz cargó hasta S/. 15 menos que la PD. La poda del Backtracking redujo los estados visitados entre 42 % y 86 % sin perder la ruta óptima. Dividir la ciudad en una zona por repartidor redujo la distancia total de 43.4 a 30.2 km.

- **Sobre el sistema:** la aplicación ejecuta todas sus funcionalidades sin bloquearse, gracias a los cálculos en segundo plano y al manejo centralizado de errores; funciona

sin conexión a Internet con un mapa vectorial propio; y permite cerrar calles y observar su efecto en todos los algoritmos. Una prueba automática de la interfaz recorre todas las pestañas y respalda esta afirmación.

8.2 Recomendaciones

- Para escalar a más de 14 paradas, sustituir Held-Karp por heurísticas de mejora como 2-opt aplicadas sobre la ruta del Greedy, que reducen el exceso de distancia en tiempo polinomial.
- Incorporar el volumen como segunda restricción de la mochila, con una tabla $dp[i][w][v]$ de costo $O(N \cdot W \cdot V)$; actualmente el volumen solo se verifica después de optimizar por peso.
- Calcular la mediana por selección lineal (Quickselect) en lugar de ordenar en cada nivel, para que la partición espacial baje de $\Theta(n \log^2 n)$ a $\Theta(n \log n)$.
- Revisar en campo los ocho lugares cuya ubicación se desplazó más de 60 m al ajustarse a la red vial (Anexo B) y reconstruir el caché vial con las coordenadas corregidas.
- Añadir persistencia de pedidos (por ejemplo, SQLite) y tiempos de viaje por tipo de vehículo, para que la herramienta pueda usarse en la operación diaria de una empresa de reparto.

Referencias

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press, Princeton, NJ.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4 edition.

Dantzig, G. B. and Ramser, J. H. (1959). The truck dispatching problem. *Management Science*, 6(1):80–91.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271.

Held, M. and Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210.

Rosenkrantz, D. J., Stearns, R. E., and Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581.

Toth, P. and Vigo, D. (2014). *Vehicle Routing: Problems, Methods, and Applications*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2 edition.

Apéndice y Anexos

Anexo A: Estructura del Repositorio

```
proyecto-progra-iii/
|-- main.py                  # Punto de entrada de la aplicacion
|-- config.py                # Parametros globales (mapa, colores, limites)
|-- requirements.txt         # Dependencias de la aplicacion
|-- requirements-dev.txt     # Dependencias para reconstruir la red vial
|-- core/                    # Algoritmos y modelo (sin interfaz grafica)
|   |-- modelos.py           # Nodo, Arista, Pedido, Repartidor, Ruta, Grafo
|   |-- grafo.py             # Dijkstra, matriz de distancias, caminos
|   |-- geo.py               # Haversine y asignacion al lugar mas cercano
|   |-- estructuras.py       # Merge/Quick Sort, busqueda binaria, indices
|   |-- greedy.py            # Vecino mas cercano, emergencia, mochila voraz
|   |-- divide_venceras.py   # Particion en zonas de K y en R zonas
|   |-- prog_dinamica.py     # Mochila 0/1 y Held-Karp
|   |-- backtracking.py      # Rutas simples con poda opcional
|   +-- gestor_datos.py      # Carga de JSON
|-- gui/
|   |-- app.py               # Ventana principal, errores y barra de estado
|   |-- tab_mapa.py          # Red vial y cierre de calles
|   |-- tab_pedidos.py       # Ordenacion, busqueda y alta de pedidos
|   |-- tab_greedy.py        # Pestana Greedy
|   |-- tab_dyv.py           # Pestana Divide y Venceras
|   |-- tab_pd.py            # Pestana Mochila y Held-Karp
```

```
|   |-- tab_backtracking.py  # Pestana Backtracking
|   |-- tab_comparacion.py   # Graficos y tabla Big-O
|   |-- componente_mapa.py   # Mapa (tkintermapview o Canvas sin conexion)
|   |-- canvas_fallback.py   # Mapa vectorial sin conexion
|   |-- hilos.py             # Calculos en segundo plano
|   +-- panel_desplazable.py # Paneles laterales con desplazamiento
|-- data/                    # red_vial_cache.json, pedidos.json, repartidores.json
|-- benchmarks/              # benchmark.py y resultados_benchmark.json
|-- tests/                   # 39 pruebas pytest
|-- tools/                   # Construccion de la red vial y prueba de la GUI
+-- informe/                 # Este informe (LaTeX) y generar_figuras.py
```

Anexo B: Lugares de Interés de la Red Vial

Coordenadas ajustadas a la intersección vial más cercana (OpenStreetMap, 19/10/2026). La última columna indica cuánto se desplazó cada lugar respecto de su coordenada original; * marca desplazamientos mayores a 60 m, pendientes de revisión.

Tabla 8.1
Lugares de interés del grafo vial de Cusco.

ID	Nombre	Sector	Lat	Lon	Ajuste (m)
N01	Plaza de Armas	Centro Histórico	−13,51653	−71,97961	89.8*
N02	San Blas	Centro Histórico	−13,51503	−71,97478	30.5
N03	Coricancha	Centro Histórico	−13,51991	−71,97561	64.6*
N04	Mercado San Pedro	Centro Histórico	−13,52041	−71,98316	117.7*
N05	Plaza Santiago	Santiago	−13,52327	−71,98586	34.0
N06	Santa Ana	Santiago	−13,51760	−71,98823	27.5
N07	Sacsayhuamán Acceso	Centro Histórico	−13,50771	−71,98014	267.9*
N08	Limacpampa	Centro Histórico	−13,51943	−71,97362	123.0*
N09	Lucrepata	Wanchaq	−13,51789	−71,96903	13.0
N10	Av. El Sol (Cruce Pardo)	Wanchaq	−13,52348	−71,97196	5.3
N11	Wanchaq (Estación)	Wanchaq	−13,52475	−71,96853	7.1
N12	Plaza Túpac Amaru	Wanchaq	−13,52264	−71,96374	30.0
N13	Ttio La Florida	Wanchaq	−13,53055	−71,95937	14.9
N14	Hospital Regional	Wanchaq	−13,52511	−71,95612	45.4
N15	Av. Cultura (UNSAAC)	Wanchaq	−13,52698	−71,95139	11.8
N16	Cuatro Torpes	San Sebastián	−13,53142	−71,94814	66.6*
N17	Magisterio	San Sebastián	−13,52840	−71,94183	21.5
N18	Marcavalle	San Sebastián	−13,52984	−71,93343	46.5
N19	Plaza San Sebastián	San Sebastián	−13,53256	−71,92627	76.5*
N20	San Jerónimo Entrada	San Sebastián	−13,54129	−71,90449	64.1*

Anexo C: Resultados de Pruebas Unitarias

```
$ pytest -v
tests/test_backtracking.py::test_backtracking_dos_rutas_y_optima PASSED
tests/test_backtracking.py::test_backtracking_con_calle_bloqueada PASSED
tests/test_backtracking.py::test_backtracking_sin_conexion PASSED
tests/test_backtracking.py::test_backtracking_poda_por_costo PASSED
tests/test_backtracking.py::test_sin_poda_enumera_todas_las_rutas_san_blas_wanchaq PASSED
tests/test_backtracking.py::test_sin_poda_respeta_calles_cerradas PASSED
tests/test_dyv.py::test_particion_tamanios_menores_o_iguales_a_k PASSED
tests/test_dyv.py::test_resolver_rutas_dyv_ensamble PASSED
tests/test_dyv.py::test_particion_en_r_zonas_exactas PASSED
tests/test_dyv.py::test_un_repartidor_por_zona PASSED
tests/test_dyv.py::test_held_karp_por_zona_no_peor_que_greedy PASSED
tests/test_estructuras.py::test_merge_sort_numeros PASSED
tests/test_estructuras.py::test_merge_sort_reverse PASSED
tests/test_estructuras.py::test_merge_sort_pedidos_por_valor PASSED
tests/test_estructuras.py::test_quick_sort_numeros PASSED
tests/test_estructuras.py::test_quick_sort_con_duplicados PASSED
tests/test_estructuras.py::test_quick_sort_pedidos_por_prioridad PASSED
tests/test_estructuras.py::test_busqueda_binaria_exito_y_fallo PASSED
tests/test_estructuras.py::test_busqueda_binaria_en_pedidos PASSED
tests/test_estructuras.py::test_indices_hash_e_invertido PASSED
tests/test_grafo.py::test_dijkstra_reconstruye_optimo_en_grafo_pequeno PASSED
tests/test_grafo.py::test_geometria_de_ruta_concatena_curvas_en_sentido_correcto PASSED
tests/test_grafo.py::test_asignacion_de_coordenada_elige_nodo_mas_cercano_por_haversine PASSED
tests/test_grafo.py::test_cierre_actualiza_matriz_y_puede_hacer_inalcanzable PASSED
tests/test_grafo.py::test_cierre_bidireccional_cierra_arcos_en_ambos_sentidos PASSED
tests/test_grafo.py::test_cache_es_fuertemente_conexo_y_cumple_desigualdad_triangular PASSED
tests/test_grafo.py::test_cache_aristas_tienen_peso_mayor_que_recta_y_geometria_con_extremos PASSED
tests/test_greedy.py::test_vecino_cercano_orden_esperado PASSED
tests/test_greedy.py::test_vecino_cercano_con_retorno PASSED
tests/test_greedy.py::test_vecino_cercano_respeta_distancias_asimetricas PASSED
tests/test_greedy.py::test_asignacion_emergencia_mas_cercano PASSED
tests/test_greedy.py::test_asignacion_emergencia_filtro_por_capacidad PASSED
tests/test_pd.py::test_mochila_caso_clasico_libro PASSED
tests/test_pd.py::test_mochila_capacidad_cero_o_vacio PASSED
tests/test_pd.py::test_held_karp_tsp_triangulo_retorno PASSED
tests/test_pd.py::test_held_karp_tsp_4_puntos PASSED
tests/test_pd.py::test_mochila_pesos_decimales_no_exceden_capacidad PASSED
tests/test_pd.py::test_mochila_datos_reales_respetan_capacidad PASSED
tests/test_pd.py::test_mochila_coincide_con_fuerza_bruta PASSED

============================== 39 passed ==============================
```

Anexo D: Capturas de la Interfaz Gráfica

Capturas tomadas por `tools/prueba_gui.py` en modo sin conexión (mapa vectorial en `Canvas`). Con conexión a Internet, el mapa muestra además los tiles de OpenStreetMap.

Figura 8.1
Pestaña Red Vial: los 20 lugares y las conexiones por calles, con controles para cerrar calles y tabla de desvío (línea recta frente a calles).

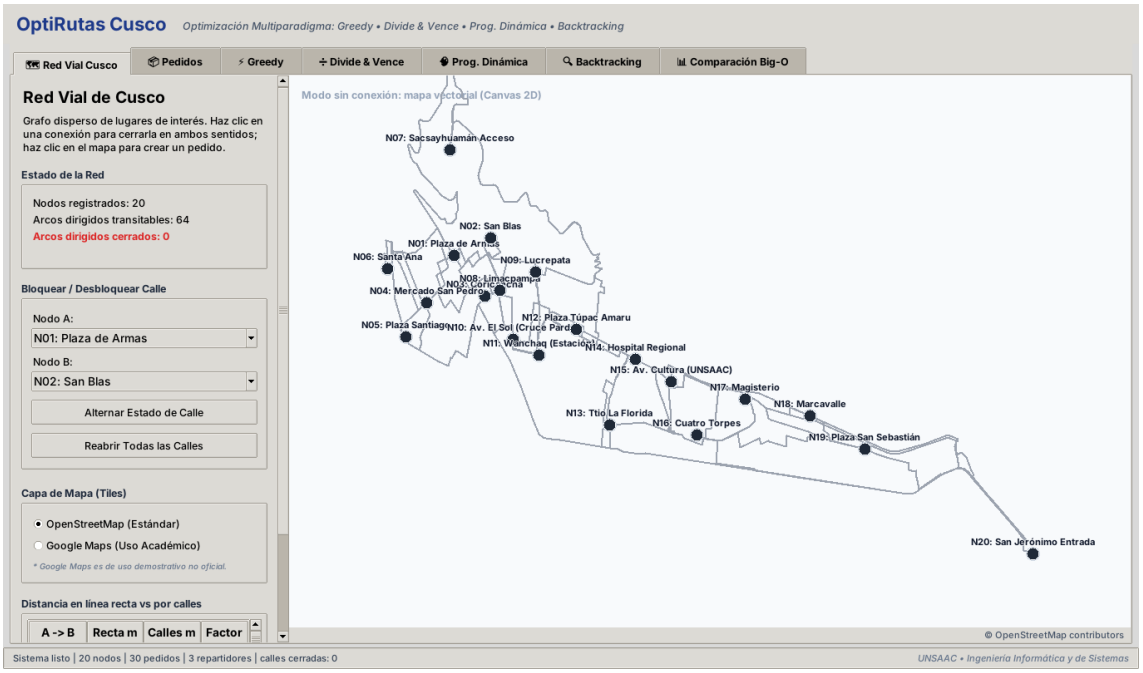


Figura 8.2

Pestaña Pedidos: pedidos ordenados por prioridad con Merge Sort manual; colores por urgencia y búsqueda por ID o sector.

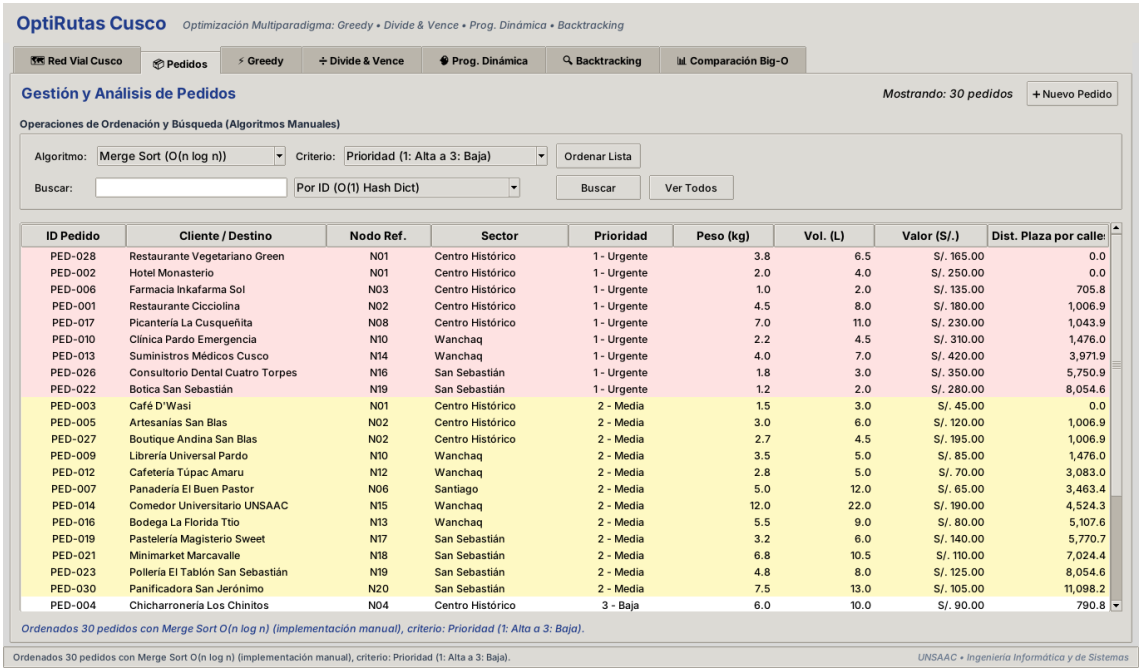


Figura 8.3
Pestaña Greedy: ruta del vecino más cercano para 8 pedidos, con las entregas numeradas en el orden elegido.

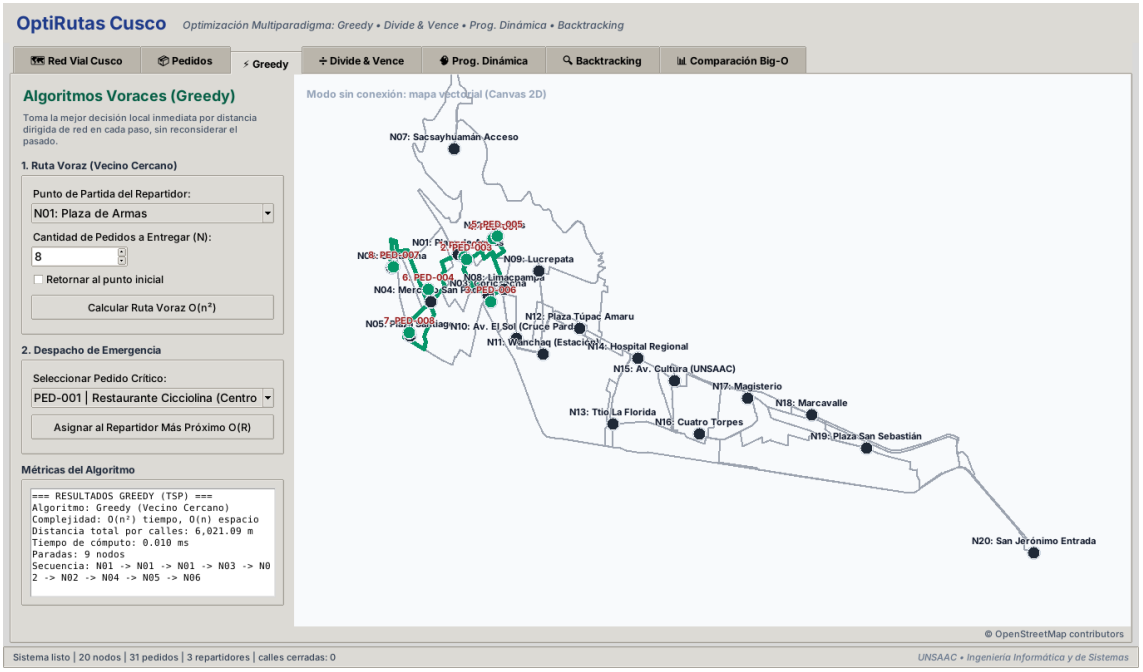


Figura 8.4
Pestaña Divide y Vencerás: tres zonas, una por repartidor, cada una con su ruta óptima.

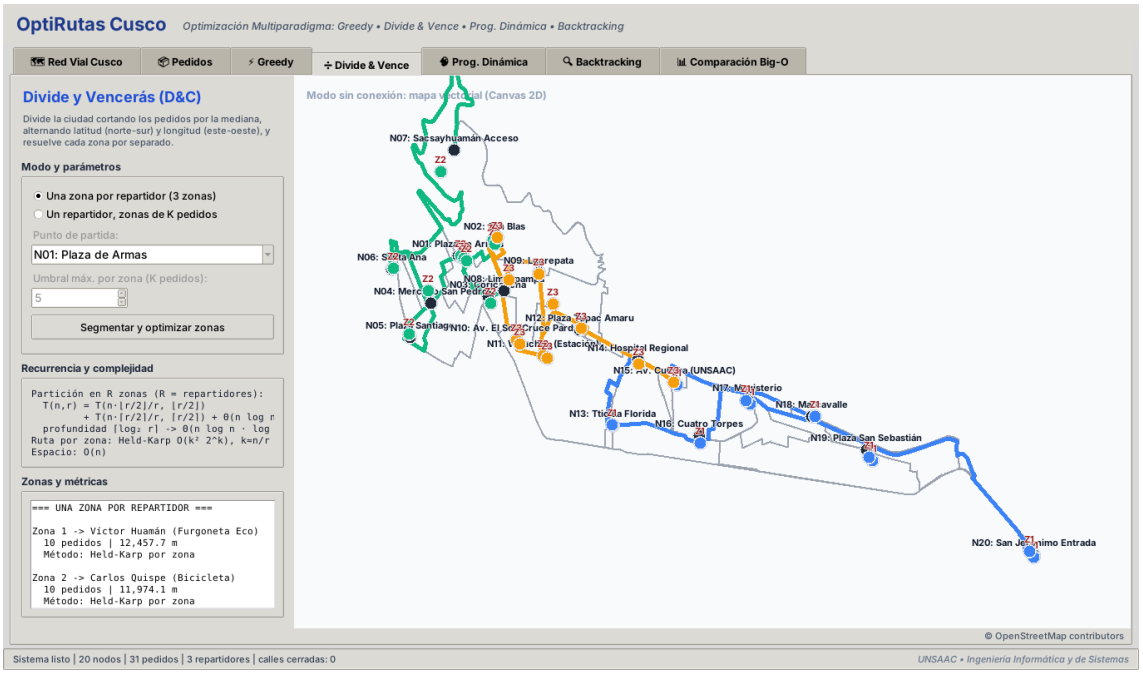


Figura 8.5
Pestaña Programación Dinámica: Held-Karp (púrpura) frente a Greedy (verde) sobre los mismos 8 puntos.

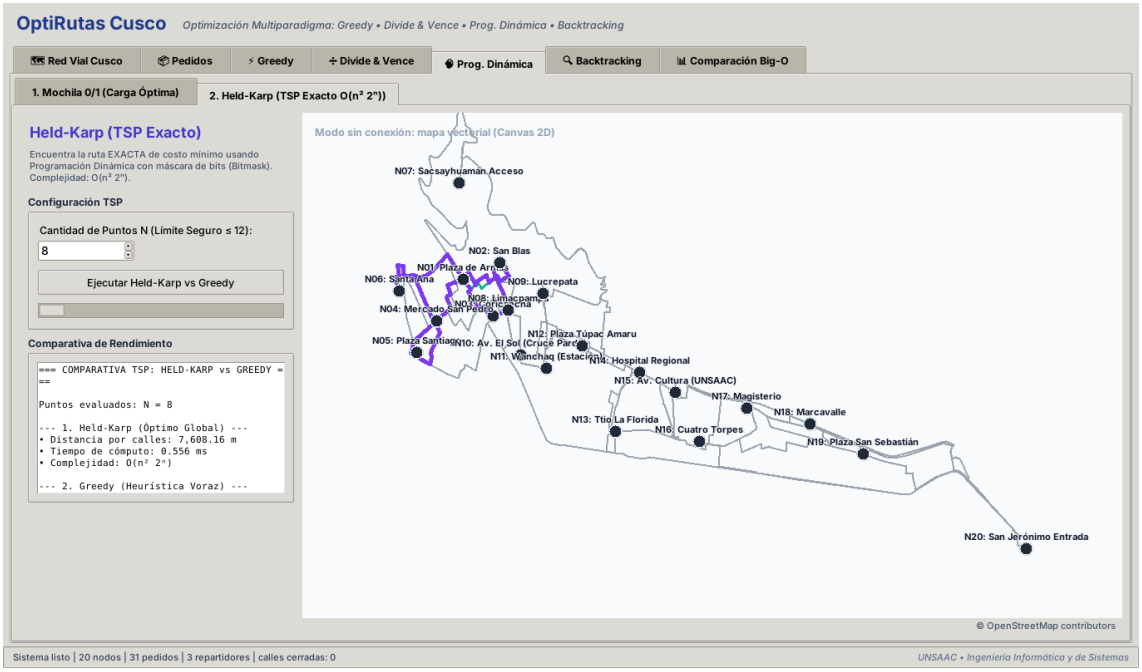


Figura 8.6
Pestaña Backtracking: San Blas a Wanchaq con la calle N02–N08 cerrada (rojo) y la nueva ruta más corta (azul).

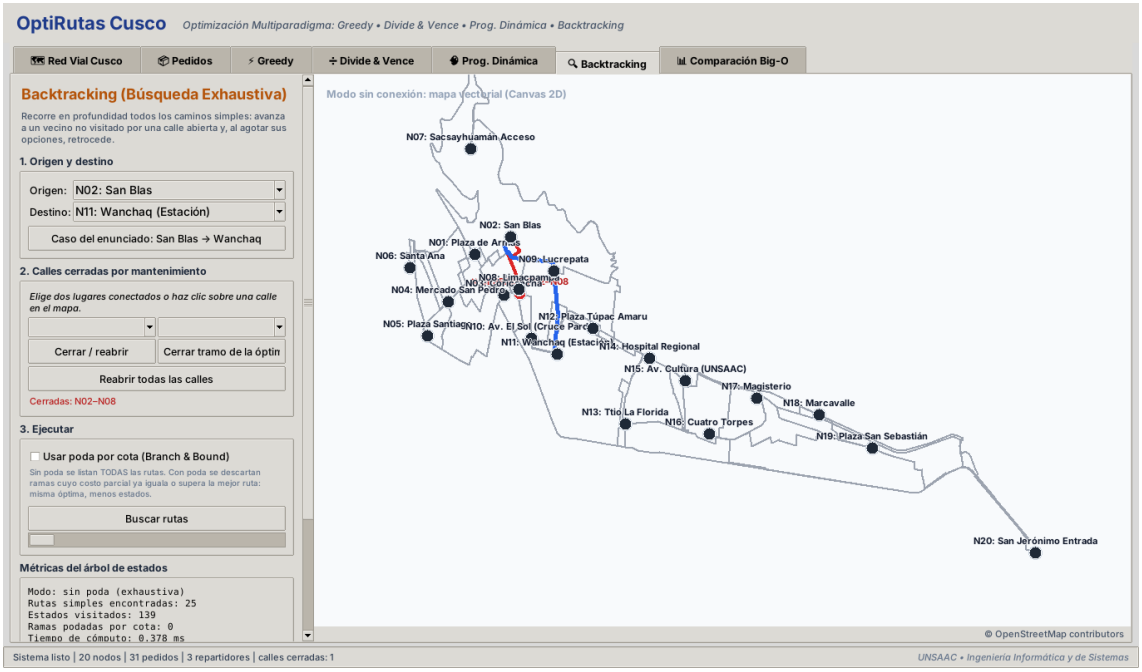


Figura 8.7
Pestaña Programación Dinámica: Mochila 0/1 con la capacidad de la bicicleta, tabla DP con los pedidos elegidos resaltados y comparación con el voraz.

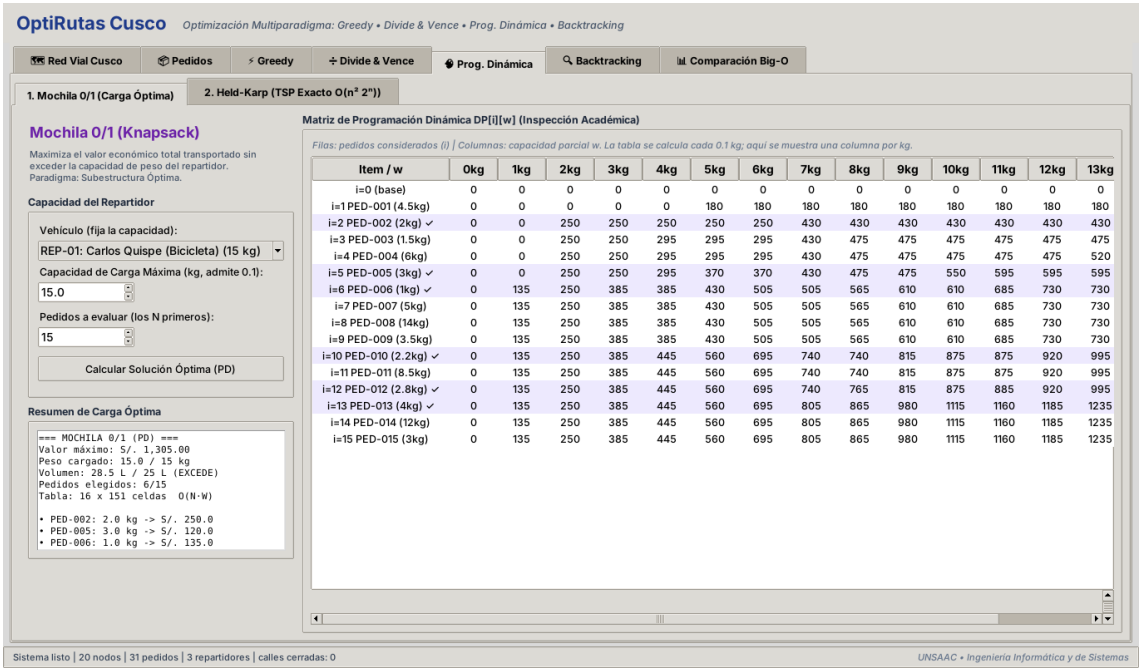
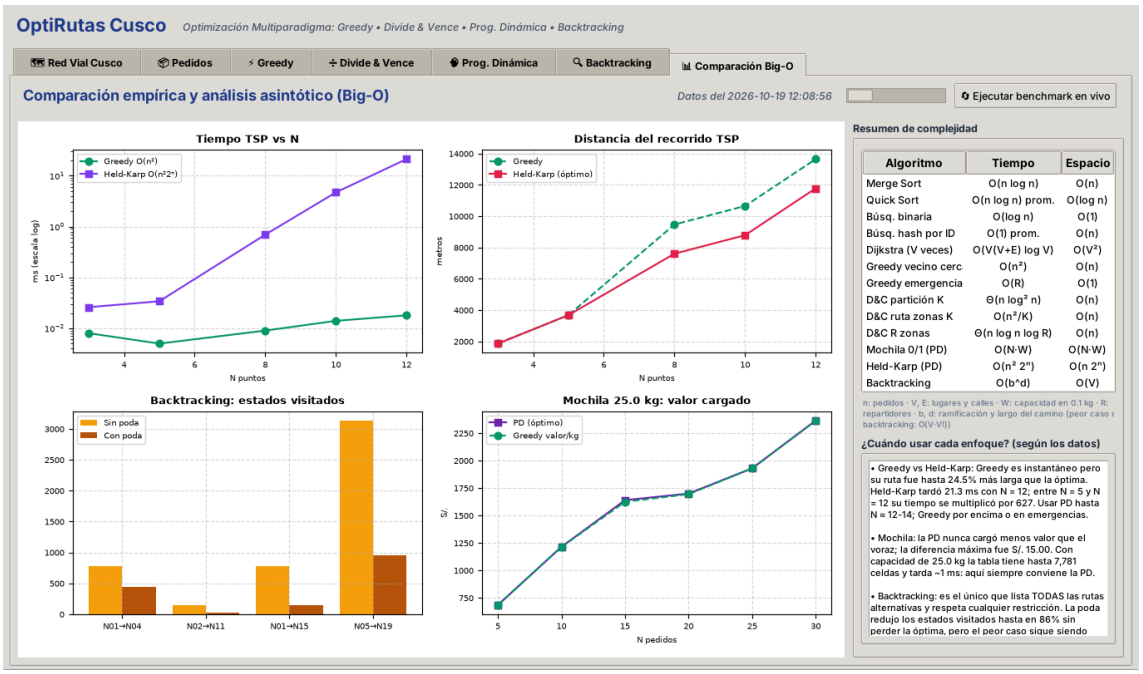


Figura 8.8
Pestaña Comparación: gráficos de los benchmarks, tabla Big-O y conclusiones calculadas a partir de los datos.



Anexo E: Divide y Vencerás con un Repartidor (Pedidos Sintéticos)

Pedidos sintéticos (semilla 2026) ubicados en los 20 lugares con una pequeña variación de coordenadas; partida desde la Plaza de Armas y $K = 5$.

Tabla 8.2
Divide y Vencerás ($K = 5$) frente a Greedy global con un repartidor.

n	**D&C (ms)**	**Greedy (ms)**	**Distancia D&C (m)**	**Distancia Greedy (m)**
30	0.215	0.054	40,912	40,715
60	0.579	0.160	53,636	43,353
120	1.240	0.599	57,637	43,353
240	3.527	2.919	75,611	43,353
480	11.581	10.492	106,593	43,353